

Section 1: About You

Goals and Objectives

Why do you want to do a GSoC project with Oppia?	From my initial introduction to the Oppia community, it quickly became more than just a place to contribute—it became a place to learn, grow, and connect with like-minded individuals. What started as an opportunity to refine my technical skills soon evolved into a deeper engagement with Oppia’s mission and its vibrant community. Why GSoC with Oppia? Community & Collaboration: Oppia stood out for me among open-source communities with its well-organized workflow, engaging weekly meetings, and highly supportive Android team. Their timely guidance and reviews on PRs, detailed Wiki, and active GitHub discussions create an environment that fosters both learning and meaningful contributions. Skill Growth & Alignment: Working on Oppia has not only aligned with my existing skills but has also been a continuous source of learning. Every deep dive into its codebase has introduced me to new technologies and best practices, strengthening my expertise in Android development. Through GSoC 2025, I see an incredible opportunity to contribute to a project that will have a lasting impact on Oppia while pushing my technical and problem-solving abilities even further. I am excited to take on a challenge that not only benefits the community but also helps me grow as a developer.	
What do you hope to learn/achieve during GSoC?	During GSoC, I hope to gain valuable experience and expand my skill set in meaningful ways. Through my past contributions, I have developed a strong foundation in writing tests, working on issues related to developers, and building user interfaces in Kotlin for end-user applications. However, with the project I am aiming for, I am particularly excited to explore more complex, developer-centric skills. I am eager to explore areas such as automation scripts, Bazel, and CI/CD workflows, while also deepening my understanding of Android command-line tools and their underlying architecture. Contributing to this project will not only enable me to make meaningful contributions to Oppia but also provide a valuable opportunity to enhance my problem-solving skills. By working across various categories of Android-related issues, I aim to gain deeper insights into improving codebase quality and further strengthen my Android development expertise.	
Which Oppia teams have you collaborated with, and what have you done on those teams?	Developer Workflow - Android	PR: #5723 Reviewed the PR for the Wiki update on onboarding instructions and Bazel documentation, focusing on: • Clarity and paraphrasing for better readability • Ease of following the instructions • Completeness and grammatical accuracy

		Reviewed the PRs (#5710, #5682) related to issue #5661 • Ensured the PR description passed the essential checklist. • Provided in-line comments to review code changes and resolved them accordingly. PR #5674: clarified PR status. Created issues #5661 #5645
Contact information	Name: Manas Gupta Preferred Name: Manas Email: manasgupta733@gmail.com LinkedIn: https://www.linkedin.com/in/manas733/	
Preferred method of communication	Google Chat Email Slack	
Social media handles (optional) <i>This field is optional. We ask for it so that we can tag you in GSoC-related posts if your application is accepted!</i>	LinkedIn: https://www.linkedin.com/in/manas733/	
Which timezone will you primarily be in during the summer?	Indian Standard Time (IST) - GMT+5:30	
(If you are a student) When are your school holidays?	I'm a college student and my summer vacation runs from May 3 to July 29. Additionally, when classes are in session, I have holidays on Saturdays and Sundays, with a half-day on Fridays.	
What other obligations might you need to work around during the summer?	During the summer vacation, I have a small college group project, but I can manage it easily as I will be free all day, and the workload is shared among group members. Once classes resume, attendance is not strict, so I can manage my schedule effectively without any conflicts. The timetable for next semester's classes, which begin in August, hasn't been released yet. However, based on previous semesters, classes typically run from 9 AM to 4 PM, including breaks (6-7 hours). Accounting for an additional 3-4 hours for miscellaneous tasks, I anticipate having around 6-7 hours of free time each day. Moreover, the college exams do not overlap with the project timeline.	
Planned time commitment	I have planned to allocate approximately 5 to 6 hours per weekday during my summer vacation (May to July), ensuring consistent progress. After my classes resume (August to September), I will dedicate around 4 hours on	

	weekdays to maintain continuity. In total, committing ~380-390 hours to the project. I am enthusiastic about my contributions and staying actively engaged even on weekends, as it helps me remain efficient and avoid the disconnect that can come from long gaps. I am also flexible to adjust my schedule if needed to ensure steady contributions.
--	---

Project Timeframe

Note: Oppia will only be offering a single GSoC coding period timeframe this year, starting on **Jun 2**. All work for Milestone 1 must be completed and submitted by **Jul 4** for internal feedback (with any revisions due by **Jul 18**), and all work for Milestone 2 must be completed and submitted by **Aug 27** for internal feedback (with any revisions due by **Sep 10**). We will not be able to extend these deadlines.

Coding period	I will adhere to the above deadlines.
---------------	---

Communication Channels

Note: The Oppia team places a high emphasis on communication, and we have found that daily contact between contributors and mentors is important for helping keep projects on track. This is why we ask that contributors send short daily updates to their mentors explaining what they have done, where they are stuck, and what they plan to do next.

I can commit to sending daily updates to my mentor by email, each day I work during the GSoC period.	Yes
In addition to the above: how often, and through which channel(s), do you plan on communicating with your mentor?	I would prefer to stay connected at least once a day, preferably towards the end of the day, to share my progress with my mentor. Additionally, I plan to provide a structured weekly update for a designated day of the week to summarize key developments, challenges, and next steps. I have been communicating with the Android team via Google Chat and am open to using any preferred medium. Previously, I have used Slack and email, so those options would also work well. Regular communication will help me stay accountable, address challenges promptly, and ensure smooth progress throughout the project.

Contributor Briefing

Note: A couple of days after the selection results are announced, there will be a mandatory contributor briefing for accepted contributors. Please confirm whether you will be able to attend the briefing.

If accepted, I will be able to attend the contributor briefing at 3 pm UTC on May 10.	• Yes
---	---

Section 2: About Your Project

Project Details

Project title <i>(should match the one on the Project Ideas list)</i>	4.3. Android lint infrastructure and fixes
Project size <i>(should match the options on the Project Ideas list)</i>	Large (~350 hours)
Why did you choose this project?	I chose this project mainly for two reasons: • Firstly, my prior work with Kotlin script, XML parsers, and writing tests for scripts while contributing to Oppia with many developer-focused contributions prepared me for the challenges this project presents. • Secondly, the curiosity about how Android command-line tools function and the problem-solving aspect of this project excites me, as it allows me to apply my existing knowledge while continuously learning and tackling new challenges. This perfect blend of familiarity and exploration makes the project even more compelling. Spending time understanding the project's workflow and technical intricacies has further reinforced my enthusiasm. And I am eager to contribute meaningfully and make a lasting impact through this project.

Required Skills (Project-Specific)

GENERAL	
I can figure out the root cause of an issue, and communicate it effectively using a debugging doc . ¹	Debugging Doc related to PR #5677: 📄 [SOLVED][2025] Debugging Doc for #5677 Reducing Long Pres...

¹ To develop this skill: Tackle an issue where the root cause is unclear, identify the root cause in a comment on the issue thread (perhaps with a debugging doc explaining how you figured it out) and submit a PR that fixes it. Fixing CI failures also counts. You might find [this tutorial](#) helpful.

I can debug and fix CI failures.²	PR #5574 – Investigated and debugged the root cause of the code coverage comment failing due to a disallowed conclusion. Provided relevant test PRs and workflow runs to facilitate discussion and identify a resolution.
I can make changes to GitHub Actions workflows.³	PR #5574 – Updated <code>comment_coverage_report.yml</code> to notify first-time contributors that manual approval is required. PR #5603 – Upgraded the version of <code>actions/cache</code> after reviewing its migration guide. Provided a relevant test PR and a workflow run on the <code>develop</code> branch as proof for changes made in both the above PRs. PR #5599 – made changes to <code>static_checks.yml</code> for script integration into the workflow.
ANDROID	
Build the app and install it on a local device or emulator. Then verify that you can (a) build a non-test library target locally, (b) run a unit test locally, and (c) play through the app locally.⁴	GitHub Gist for bazel build: https://gist.github.com/manas-yu/0ca3d71b3c693ee1b9eda4395382aa60 Screenshot of running <code>StateFragmentTest</code>: https://drive.google.com/file/d/1wf6y5nq1T6SF935Yo7IOsBjTebByHvsh/view?usp=sharing Video recording of Prototype lesson playthrough: https://drive.google.com/file/d/1FbUjWKp7qNiDrnOcZZ-LUDh5NBCjz26j/view?usp=sharing
Write new Kotlin code with unit tests.⁵	PR #5599 – Developed the <code>TextViewStyleCheck</code> automation script in kotlin, along with its tests, to enforce the use of centrally managed styles for all <code>TextView</code> elements in layout XML files. Implemented XML parsing using the SAX parser. PR #5742 – Wrote tests for build stats utility scripts (Though this PR is not merged, I wanted to mention this since it's relevant to the project). PRs #5614 #5655 #5704 #5691 – Introduced a custom generation pipeline for content descriptions with its integration into respective sections of the app, enhancing accessibility.

² To develop this skill: See issues labelled “[CI breakage](#)” on GitHub, or look at [CI failures in develop](#) (or [✗ signs here](#)) and figure out how to fix them. [This tutorial](#) (for Web) might also be useful.

³ To develop this skill: Please see the list of issues for the [dev workflow team](#). However, note that this skill is not needed for most projects and there are limited issues available for it. If this affects your project, [please contact the org admins](#).

⁴ To develop this skill: Please follow the [Bazel setup instructions](#) to get your local setup working. If you run into issues, ask a question in [Oppia Android's discussions board](#).

⁵ To develop this skill: essentially all [CLaM issues](#) will require Kotlin and testing work. Focus on issues that aren't marked as “backlog”.

	PR #5627 - Wrote tests for math utilities along with their Bazel build file. PR #5622 - Validated Kdoc ending sequences through regex pattern validation script. PR #5550 - Fixed consoleLogger overwriting local log file bug.
Change Android UIs, write tests for them, and manually verify that they work.⁶	PR #5687 - Fixed rendering of images in list items, ensuring they render as block elements PR #5677 - Reduced the Long Press time in Drag-Drop interactions PR #5593 - Included caption-with-value handling in ImageTagHandler and ensured proper rendering of captions. PR #5647 - Fixed vertical alignment of locally rendered and cached LaTeX

(Optional) Oppia Community References

Who you worked with	Context in which you worked with them
Adhiambo Peres (@adhiamboperes)	Review, approval, and discussion of all my contributions.
RD (@Rd4dev)	Guidance and discussions on Developer-Workflow issues.
Ben Henning (@BenHenning)	Debugging and Problem Solving.

Section 3: Proposal Details

Problem Statement

Target Audience	Developers
Core User Need	As a developer contributing to Oppia, I need an automated and efficient way to

⁶ To develop this skill: Any CLaM issue that involves fixing something that can be observed from within the app itself will apply to this skill. See [this list](#) as a high-level reference (but note that some of those only affect tests). If you're unsure whether a particular issue applies, ask in the Oppia Android GSoC Q&A Discussion board.

	detect and fix Android-specific code issues without relying on Gradle so that I can maintain high code quality and ensure my changes meet Oppia's standards. Additionally, I need a clear exemption system to manage known issues and a well-documented process to easily integrate lint checks into my workflow.
--	---

Section 3.1: WHAT

Key User Stories and Tasks that will be implemented

#	Title	User Story Description (role, goal, motivation) <i>"As a ..., I need ..., so that ..."</i>	List of tasks needed to achieve the goal (this is the "User Journey")	Links to mocks / prototypes, and/or PRD sections that spec out additional requirements.
1	Target Specific linting analyses	As a developer, I need to run Android Lint so that I can detect and fix Android-specific issues efficiently.	Execute the Android Lint script with the required arguments and targets to customize the linting process.	
			Review the generated lint reports to identify issues and understand their impact.	
			Locate and resolve the detected lint issues.	
			Re-run the script to ensure it passes successfully.	
2	Automate Linting Analyses in CI workflow	As a developer, I need automated lint checks in CI so that I can ensure code quality and prevent regressions.	Make changes locally , then commit and push them to the origin branch.	
			Analyze the Android Lint script report in the static checks workflow to verify compliance.	
3	Support Exemptions for Android Lint	As a developer, I need to add exemptions for specific Android Lint failures so that I can temporarily exclude known issues while ensuring new ones are properly addressed.	Run the Android Lint script to generate a report of detected issues.	
			Locate the reported lint issue and its corresponding file in the generated report.	

			Update the textproto exemption list with the appropriate proto message format and	
			Re-run the script to ensure it passes successfully.	

Testing Plan

Critical user journeys

User Type	User Story Description	Test definition		
		Setup (ends with a user of the correct type getting logged in)	Test steps	Test expectations
Developer	Run Android Lint checks locally using the Android Lint script.	- Set up Oppia-Android and bazel. - Execute the Android Lint Script through “bazel run” for linting analysis. - Review the Android Lint Analysis report.	Execute the script with “bazel run //scripts:run_lint -- \$(pwd)”	The script successfully runs and analyzes lint issues for modules (/app,/utility,/domain,/testing,/data), generating a comprehensive project-level report.
			Execute the script with project module(s) having lint issues.	The script effectively interprets and catches issues of broader groups across all modules, including Code Quality (Correctness, Productivity, Interoperability), Security , Performance , User Experience & Accessibility (Usability, Accessibility, Internationalization), Testing , also differentiating them by severity levels: Fatal, Error, Warning, and Informational.

			Execute the script with a lint issue categorized under Lint Implementation with a severity of Error or Fatal.	The script fails with an <code>IllegalStateException</code> , indicating an internal lint issue.
			Execute the script with project module(s) having lint issues with a severity of "Error/Fatal."	The script terminates with a "Fail" status after logging issue IDs along with their location, description, priority, and severity.
			Execute the script with project module(s) having lint issues with a max severity of "Warning."	The script executes successfully with a "Pass" status after logging issue IDs along with their location, description, priority, and severity.
	Use a textproto file to override specific lint failures.	• Set up Oppia-Android and bazel. • Navigate to scripts/assets. • Update the lint textproto with the lint issue to be exempted • Execute the Android Lint Script to review reports	Execute the script with project module(s) containing an issue ID with the severity "Error/Fatal" in files A and B.	The script terminates with a "Fail" status, logging the issue ID in both files.
			Execute the script with project module(s) containing an issue ID with the severity "Error/Fatal" in files A and B, where the exemption textproto lists file A for the corresponding issue ID.	The script terminates with a "Fail" status, logging the issue ID for file B while exempting it for file A.
			Execute the script with project module(s) that includes a lint issue ID with severity "Error" or "Fatal" which is not defined in the Issue ID enum in	The script should terminate with a "Failure" status, clearly logging the issue ID and prompting the user to: Add the missing issue ID to the Issue ID enum in <code>script_exemptions.proto</code>.

			script_exemptions.proto.	Update the enum-to-ID mapping logic in the script accordingly. Update the textproto exemption list with the appropriate message format.
	Run the script as part of CI to enforce linting on PRs.	- Make changes to the cloned oppia-android repository. - Create a PR pushing changes to the develop branch. - Review the Android Lint workflow logs.	Create a test PR with a known lint issue of severity "Error/Fatal." Push changes to the same PR, adding the known issue to the exemption list.	The Android Lint workflow fails as expected, detecting the known issue. The Android Lint workflow passes, correctly exempting the lint issue.

Technical Requirements

Additions/Changes to the Web Server Interface

#	Endpoint URL	Request type (GET, POST, etc.)	New / Existing	Description of the request/response contract (and, if applicable, how it's different from the previous one). For new endpoints, include which access control decorator (e.g. "can_play_exploration", "open_access", etc.) will be used.
1.	N/A	N/A	N/A	N/A

New/Changed Calls by the Web Frontend /Android Client to the Web Server Interface

#	Endpoint URL	Request type (GET, POST, etc.)	Description of why the new call is needed, or why the changes to an existing call are needed
1.	N/A	N/A	N/A

Web Server Storage Model Additions/Changes

#	Datastore model	Description of changes	How will existing data be handled?
1.	N/A	N/A	N/A

UI Screens/Components

#	ID	Description of new UI component	i18n required?	Mock/spec links	What A11y support will be implemented?
1.	N/A	N/A	N/A	N/A	N/A

Data Handling and Privacy

#	Type of data	Description	Why do we need to store this data?	Anonymized?	Can the user opt out?	Wipeout policy	Takeout policy
1.	N/A	N/A	N/A	N/A	N/A	N/A	N/A

Connection to Existing Work

Similarity	Code Section	Usage
Wrapping of the command line tool	apkStats scripts	Wrapping the Android CLI tool with a Kotlin script to automate its execution and integration with the project.
Textproto loading	KdocValidityCheck , script_assets , script_exemptions	- Loading exemption messages from the textproto file for subsequent transformation into XML format. - Logging redundant exemption messages to suggest their removal.
XML parsing and CI integration	TextViewStyleCheck , static_checks	- Parsing elements and attribute values from the final lint analysis XML file to determine the overall result as PASS or FAIL, along with issue descriptions. - Integration into static_check CI and script.

Other Requirements

Requirement (Technologies/Tools)	Purpose
Maven Repositories	Leverage artifacts from the Maven repository to facilitate the linting process.
Bazel	Ensure that Android Lint can be executed within a

	Bazel-based environment.
ProtoBuf/TextProto	Store and manage exemption lists for lint failures in a structured format instead of XML.
XML	Process temporarily generated XML lint files to analyze reports and enforce exemptions.
CI/CD	Automate lint checks in pull requests and the main repository to maintain code quality.

Section 3.2: HOW

Existing Status Quo

The codebase incorporates several tools to enforce coding standards and maintain code quality:

- **Ktlint** – Enforces Kotlin coding conventions, ensuring proper formatting and style compliance.
- **Checkstyle** – Applies Java coding standards and best practices using configurable rule sets.
- **Buf** – Lints and formats Protocol Buffers to maintain consistency and correctness.
- **Buildifier** – Formats and lint-checks Bazel build files to uphold best practices.
- **Translatewiki.net** – Manages translations and ensures localization consistency.
- **Bazel-based script checks** – Includes validation checks such as KDoc checks, XML syntax validation, and more.

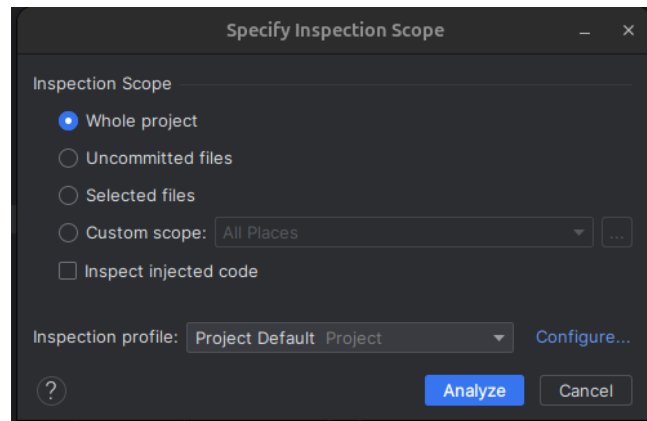
Additionally, the project uses **Android Lint**, a static code analysis tool designed specifically for Android development. It detects structural issues, potential runtime errors, usability concerns, and performance bottlenecks. Some example issues handled by Android Lint that other Linters do not cover:

- Detects misuse of Android APIs, such as deprecated methods or incorrect usage of system services.
- Warns about unused resources to reduce APK size.
- Identifies inefficient API calls, like using new SimpleDateFormat() instead of DateTimeFormatter
- Flags improper usage of Android coroutines (e.g., calling launch {} inside onCreate()).

Lint Reports:

Currently, Android Lint runs via the Gradle wrapper (./gradlew lint), exporting reports in XML and HTML formats in <module>/build/reports for modules (app, data, utility, domain, testing), with issues tracked in [#5169](#) for the app module. Below are ways to configure and use Android Lint currently in the codebase:

- Using the UI interface to run gradle lint from **Code -> inspect code..**

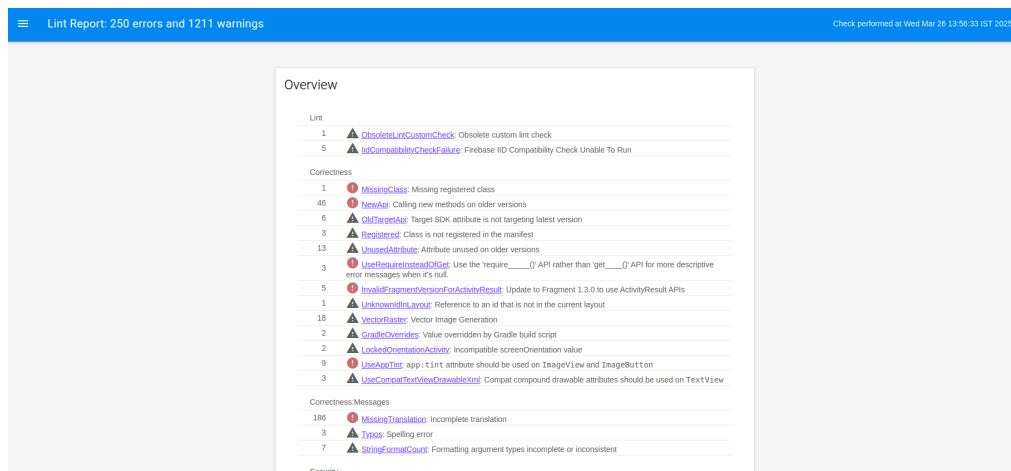


- Configuring the lint tool through [Gradle DSL](#) to customize lint checks in build.gradle by enabling/disabling specific issues and setting severity levels.
- Scoping report generation by specifying module (eg: ./gradlew :app:lint)
- Suppress warnings in code using inline annotations, such as `@SuppressWarnings("ObsoleteSdkInt")`, to selectively disable specific lint checks.

The codebase is [restricted](#) to Android Lint (version **4.2.2**), as its version is tightly coupled with the Android Gradle Plugin (AGP). This dependency presents a major limitation, as upgrading AGP can introduce compatibility challenges. Additionally, the enforced lint version is significantly outdated, with the latest stable release being **8.10.0** at the time of writing.

My attempt to generate reports via `./gradlew lint`:

App Module (HTML report):



Lint Version

Current Issues Reported

E - Errors

W - Warnings

4.2.2	App - 250 E 1211 W Data - 5 E and 131 W Domain - 24 E 183 W Testing - 25 E 216 W Utility - 5 E 128 W
-----------------------	--

Issues are reported with specific IDs and priority levels. Below are some relevant issues detected by Android Lint in the current codebase:

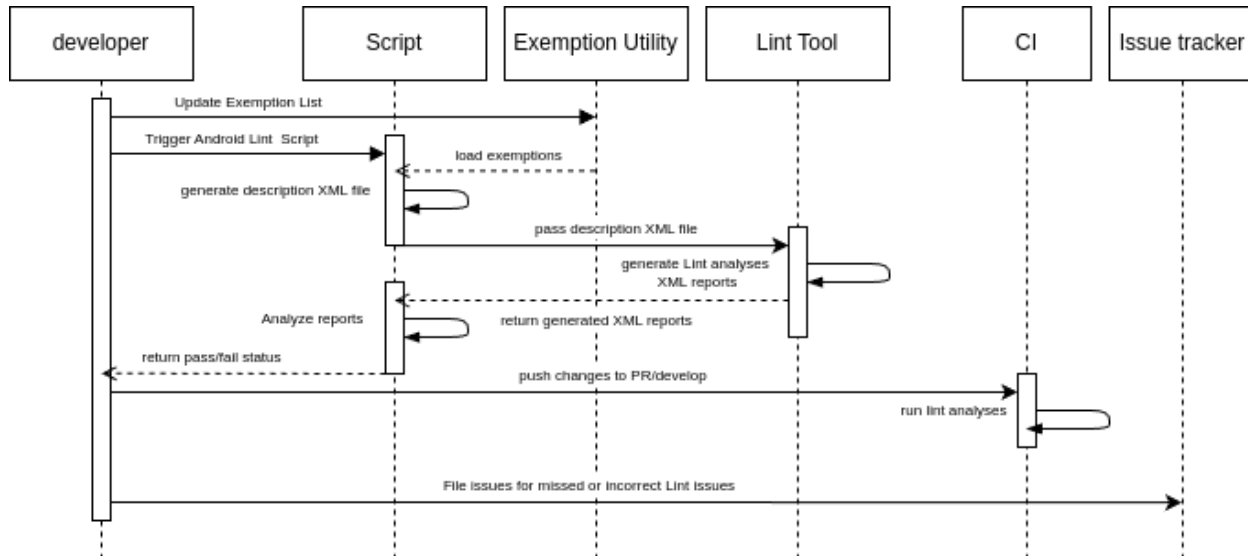
Severity Level	Issue IDs
Fatal	InvalidFragmentVersionForActivityResult
Error	Missing Class NewApi UseRequireInsteadOfGet UseAppTint
Warning	OldTargetApi UnusedResources SyntheticAccessor UnknownNullness SelectableText

- Given the intricate connection between the Lint tool and the AGP, and considering the Android team's transition from Gradle to Bazel, finding a way to execute Lint-based static analysis independently of Gradle is essential.
- The integration of the Lint tool with Bazel is still actively [discussed](#) within the community. However, the implementation of [rules_android_lint](#) provides a strong example of how to wrap and configure Lint to suit a specific codebase.
- Android Lint has not been integrated into the CI due to its lack of support in **Bazel**, with this issue being tracked in [#1742](#).
- While using a Starlark rule like `rules_android_lint` is viable, wrapping the lint tool with a Kotlin script offers better control over result processing and exemption handling and aligns with the script-based check infrastructure already being used in the codebase.
- The codebase needs a Bazel-supported script to run Android Lint with functionality and issue reporting similar to `./gradlew lint`, enabling consistent CI integration.

Solution Overview

Proposed Solution:

Android provides a standalone [command-line tool](#) for running Android Lint on non-Gradle projects. The proposed solution involves configuring this tool, wrapping it in a Bazel-compatible script with textproto-based exemption handling, and integrating it into CI to ensure consistent static analysis.



Deliverable 1.1:

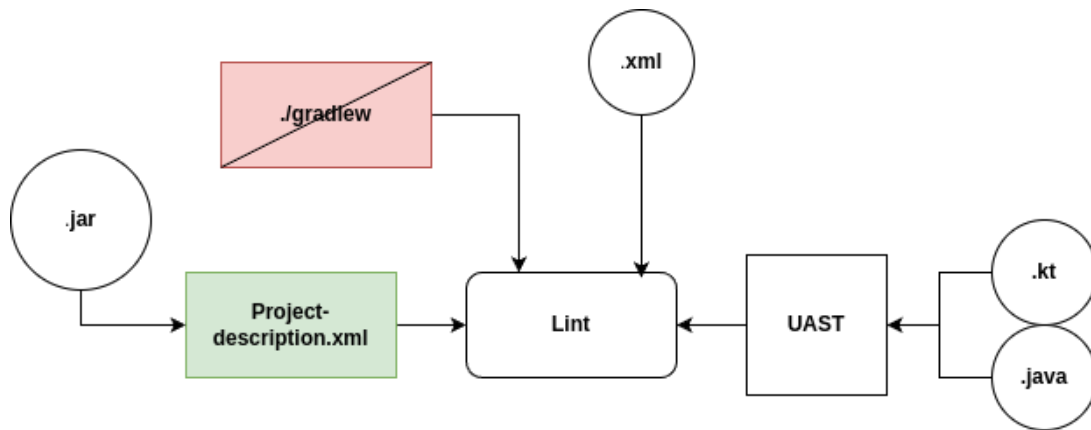
The developer triggers a script that wraps the lint command line tool, whose analysis is done below

- **Understanding Lint:**

- Gradle enhances Lint's static analysis by providing source set awareness, efficient dependency management, and flexible configuration through the [Gradle DSL](#), allowing developers to enable or disable specific checks as needed.
- The Lint tool is accessed through the script at **SDK_HOME/cmdline-tools/<version>/bin/lint**, where it ultimately runs **com.android.tools.lint.Main** using Java, passing the necessary options, with its classpath JAR files located at **SDK_HOME/cmdline-tools/<version>/lib/lint**.

```
# Collect all arguments for the java command, following the shell quoting and substitution rules
eval set -- $DEFAULT_JVM_OPTS $JAVA_OPTS $LINT_OPTS -classpath "\"$CLASSPATH\"" com.android.tools.lint.Main "$APP_ARGS"
```

- The Lint tool wraps the source sets (.kt, .java) files with a UAST (Universal Abstract Syntax Tree) for examining node files and their associated attributes.
- Further understanding of the internal workings of the Lint CLI tool can be referenced from [source code#1](#) [source code#2](#)



Lint Working

- **Configuring Lint:**

- To configure Lint to work independently of Gradle, we need to provide it with enough context to get similar results as “**./gradlew lint**”.
- Two main methods of configuring the lint command line tool:
 - [Approach 1\(not recommended\)](#) - Passing relevant context to the lint tool via flags lacks flexibility and makes configuration challenging, especially in multi-module projects. This approach also throws an [error](#) for Gradle projects, which would require removing Gradle build files.
 - [Approach 2](#) - Generating a project layout descriptor XML to define sources, resources, and libraries offers **better** configurability and scoping, making it ideal for large codebases. (The structure for the project XML file has been referenced from [source#1](#), [source#2](#), and [source#3](#))
- The Lint tool can also be configured using [Environment Variables](#) for specific checks.
- While configuring the lint tool, we specify the source sets, resource files, and JAR files that correspond to the project's dependencies, including libraries and their corresponding custom lint checks that are present in the Android archives.

- **Report Analysis:**

- The below reports are generated on branch **f6da943fb** - to maintain relevance to the technical section of the project description.
- The reports are generated using **Approach 2** above, which scopes project-description.xml at different levels by specifying the module(s) for which the lint analysis should be executed.
- Lint tool versions can be further configured using [--client-version](#). Currently, I have used only the major versions included in the command-line tools.
- Java RAM allocation was increased from default -Xmx1024m to **-Xmx2g** in “SDK_HOME/cmdline-tools/<version>/bin/lint” to avoid **OutOfMemoryError**

Cmdline-tools Version	Lint Version	Module Level Reports E - Errors W - Warnings	Project Level Reports E - Errors W - Warnings	Remarks/ Observations
				Incompatible with Kotlin (Version 1.6)

<=4.0	<=4.2.0	N/A	N/A	(terminates with: `Invalid Kotlin language level "1.6"`)
5.0	7.0.0 Kt. Ver.- 1.4.32	App - 117 E 2388 W Domain - 30 E 52 W Data - 4 W Utility - 33 E 115 W Testing - 4 E 30 W	184 E 2586 W	OutOfMemoryError for project and app-level reports. Generation of app/build/lint-resources-all.xml Increased reporting for: • NewApi • UnusedAttributes • UnusedResources • UnknownNullness No reporting of MissingTranslation Reduced reporting for SyntheticAccessor
6.0	7.1.0 Kt. Ver.- 1.4.32	App - 117 E 2378 W Domain - 30 E 52 W Data - 4 W Utility - 33 E 115 W Testing - 4 E 30 W	184 E 2579 W	
7.0	7.2.0 Kt. Ver.- 1.5.31	App - 118 E 2445 W Domain - 30 E 52 W Data - 4 W Utility - 33 E 115 W Testing - 5 E 34 W	186 E 2650 W	
8.0	7.3.1 Kt. Ver.- 1.5.31	App - 122 E 2445 W Domain - 30 E 52 W Data - 4 W Utility - 33 E 115 W Testing - 5 E 30 W	190 E 2646 W	
9.0	7.4.0 Kt. Ver.- 1.7.10	App - 122 E 2445 W Domain - 30 E 52 W Data - 4 W Utility - 33 E 115 W Testing - 5 E 30 W	190 E 2646 W	
10.0	8.0.0 Kt. Ver.- 1.7.10	App - 122 E 2442 W Domain - 30 E 48 W Data - 2 W Utility - 33 E 109 W Testing - 5 E 28 W	190 E 2629 W	
>=11.0	>8.0.0	N/A	N/A	Requires JDK 17 (terminates with: `This tool requires JDK 17 or later`)

- Although Lint version 7.4.0 reports the maximum number of lint issues, the only significant difference between 7.4.0 and 8.0.0 is the reporting of the **UsesMinSdkAttributes** issue, which has been marked as [\[Withdrawn or Obsolete Issues\]](#). Other improvements between the two versions can be referenced [\[here\]](#).
- Furthermore, **Lint 8.0.0** was built using Kotlin [1.7.10](#), whereas **Lint 7.3.1** was built with Kotlin [1.5.31](#). Since the current codebase uses Kotlin 1.6 and both Lint versions produce nearly identical results—with the only difference being the UsesMinSdkAttributes issue—it is reasonable to conclude that Lint issue reporting is not affected by the Kotlin version used to build the Lint tool,

particularly when the Kotlin version used by the project is explicitly defined in the [project description](#) or via flag (-kotlin-language-level).

Version **8.0.0** is the latest compatible version with the codebase and is, therefore, the most suitable for integration.

- **Limitations:**

Below are limitations regarding lint in general, which were noticed during the report analyses above (which utilized Approach 2)

Issue	Description	Mitigatable in Oppia (Y/N)	Relevant Discussions/Links
OutOfMemoryError	Lint hangs or exits with the error OutOfMemoryError due to misconfigurations or heavy RAM usage while running on large codebases.	Yes Addressing misconfigurations or increasing Java's RAM allocation (default: -Xmx1024m) can help resolve this issue. The above report generation used approximately 2GB of RAM. Further testing can help determine the minimum required RAM for efficient Lint execution and identify potential optimizations.	Lint Fails with OutOfMemoryError Performance Tuning
lint-resources-all.xml	Running the Lint tool generates <module>/build/lint-resources-all.xml , which it uses internally to analyze and validate resource issues.	No The lint tool does not offer a direct option to prevent the creation of this file or to specify a custom path for its generation. The file is generated even after the removal of Gradle build files or specifying the cache directory for the lint tool.	N/A
checkDependencies	checkDependencies is true by default, which works for project-level reports by utilizing <dep> tags. However,	Yes Configured Lint for project-level reports and	True by default checkDependency - need of it and its

	module-level reports also include issues from dependent modules.	maintaining the target scope for project-level reports. Running lint on the app module will also run it on all the dependent modules the app depends on – e.g. all the libraries (utility, domain, data, testing are also treated as libraries by Gradle). But this should also make things run a lot faster, since it's a single lint invocation, where lint shares a lot more computation This isn't just a good idea from a performance perspective; we will also get more accurate results. For example, the unused resource analysis will now correctly know whether a resource defined in a library is consumed by the app module.	impact on performance Project-level Reports with checkDependencies
False Positives	The lint tool heavily relies on the project-description.xml file to understand the project structure. Misconfigurations in this file lead to false positives being reported.	Yes Ensure the correct generation and validation of the project descriptor file for accurate issue reporting. Alternatively, suppress issues that result in false positives.	N/A

Run Lint Analyses:

The script will prepare an arguments list with [CLI flags](#) to configure the lint tool, including

- -Wall: check all warnings
- --quiet: Don't show progress.
- --fullpath: Use full paths in the error output. (for detailed reports)
- --xml/-sarif: Create an XML/sarif report.
- --showall: not to truncate long messages (for detailed reports)
- --project: to pass the project-description.xml

To run the Lint analysis, we pass the argument list to the Lint tool, as discussed earlier. The script wraps the main lint class, [com.android.tools.lint.Main](#), to pass the argument list and initiate lint analysis for report generation. There are two ways to invoke the main class:

- **Command line utilities:**
 - Locates the Android SDK's Lint executable and required JARs (e.g., lint-api.jar, lint-checks.jar) in the SDK installation, typically under "cmdline-tools/<version>/lib/lint."
 - Uses CommandExecutor (similar to [BundleToolClient](#)) to run "com.android.tools.lint.Main" in a separate Java process, also handling the hangs caused by non-daemon threads.
 - Computes the classpath using the SDK's JAR paths, requiring the Android command-line tools to be installed, while also implementing checks to handle missing SDK or tools.
- **Maven dependencies:**
 - Declares Android Lint libraries (e.g., lint-api, lint-checks) as Maven dependencies in Bazel, similar to [ApkAnalyzerClient](#), which uses the [apkanalyzer library](#), ensuring they are automatically included in the runtime classpath.
 - Uses reflection to invoke "com.android.tools.lint.Main" and configure flags via [LintCliFlags](#), similar to [AndroidLintCliInvoker](#).
 - Enforces timeouts and termination via [CoroutineDispatcher](#) to prevent hangs.

The approach using Maven Dependencies is preferred and also feasible, as the Jar files are published on Maven and can be bundled with the script for Lint execution.

Project Descriptor:

To initialize the lint tool correctly, the script requires the project descriptor file, which defines the project's structure and modules. Given its importance for proper lint configuration, this file will be managed through a dedicated utility that is well-documented and designed to be resilient to project changes.

- **Project-description.xml**
 - This file will be passed to the [--project](#) flag to help the Lint tool understand the project structure and layout.
 - It will be constructed based on the XML file structures referenced [source#1](#), [source#2](#), and [source#3](#) ([Link to sample XML](#)), ensuring compatibility with the Lint tool's expected format.
 - The utility would use Bazel's query language ([BazelClient](#)) to collect file paths for module dependencies, generated source files, and AAR files, ensuring proper lint configuration.
 - Proper configuration of this file is crucial, as misconfigurations could lead to false positives or prevent certain issues from being detected. The goal will be to optimize the configuration to maximize the number of issues caught while maintaining accuracy.

Deliverable 1.2:

Enhance the script with an exemption system and ensure accurate reporting of lint issues.

Exemption of Issues:

For exemption handling, the script will use the same allow-listing and deny-listing configurations as other scripts

- Leveraging a textproto format for the exemption list of issues.
- Exemptions for issue IDs will be applied on a per-file basis.
- Using enums for Lint issue IDs to ensure type safety.
- Mapping of lint issue IDs to their corresponding file paths, used to identify which issues are reported in each file.

Below is a sample representation of a textproto file.

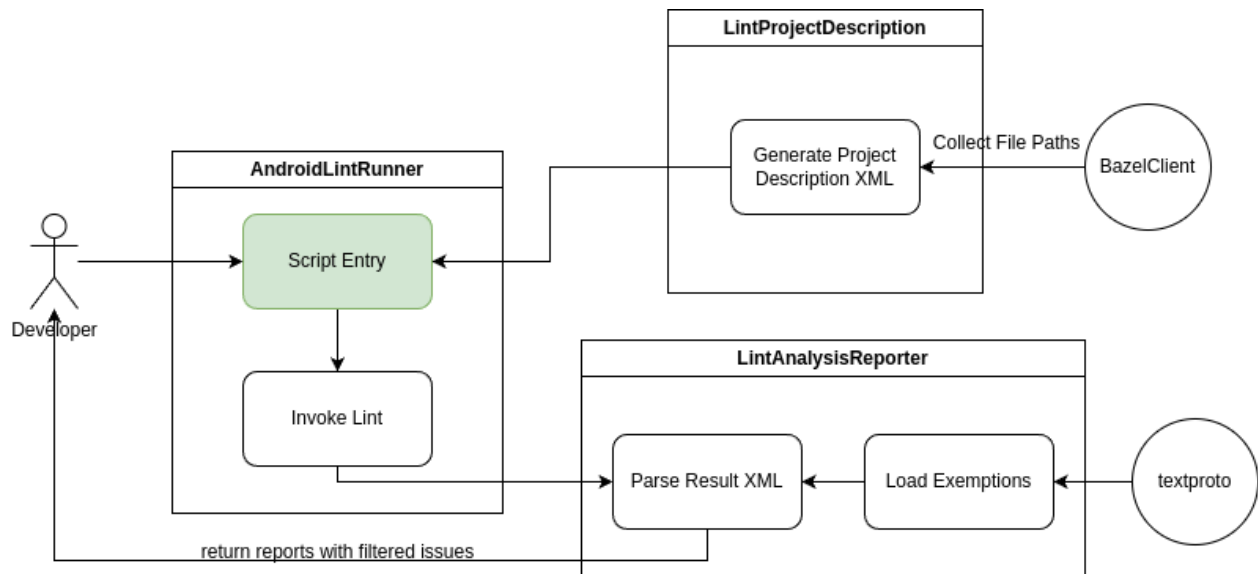
Textproto
<pre>android_lint_exemption: { exempted_file_path: "app/src/main/java/com/example/MainActivity.kt" lint_issue_id: OBSOLETE_LAYOUT_PARAM lint_issue_id: NEW_API } android_lint_exemption: { exempted_file_path: "app/src/main/res/layout/activity_main.xml" lint_issue_id: UNUSED_RESOURCES } android_lint_exemption: { exempted_file_path: "app/src/main/java/com/example/Utils/Helper.kt" lint_issue_id: NEW_API }</pre>

Report Analysis

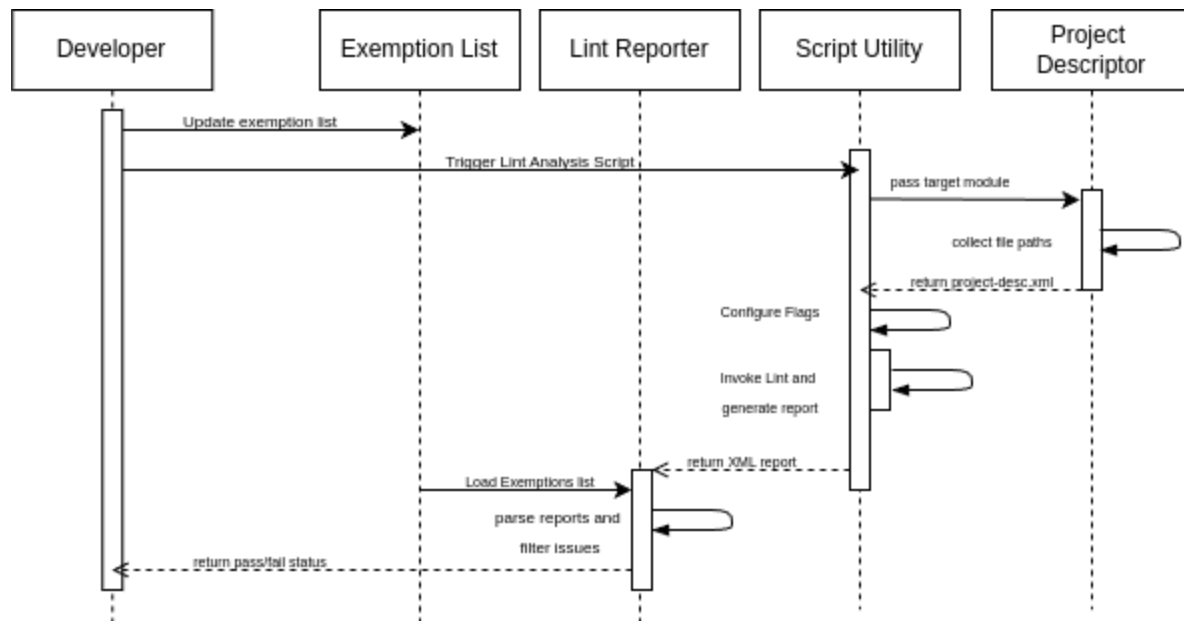
- The Android Lint tool supports **XML** and **SARIF** report generation, which will be parsed to determine pass/fail status.
- Additionally, the script will log issues along with their details, including issue ID, description, priority, severity, category, and location.

Text-based output is not considered for parsing, as it may change in future versions of the tool.

Implementation Overview:



Script Workflow Overview:



Deliverable 2.1:

Resolving the issues reported by the script to improve code correctness and maintainability.

Issues detected by Android Lint will be addressed based on their severity level and the priority ratings assigned to each [issue](#) by the Lint tool (can be referred to from generated reports), ensuring that the most critical problems are resolved first. [\[issue documentation\]](#)

Issue ID	Description	Approach	Relevant links
InvalidFragmentVersionForActivityResult Priority - 5/10 Severity - Fatal Category - Correctness	This issue occurs because registerForActivityResult() requires Fragment version 1.3.0 or higher. However, libraries like AppCompat may pull in older versions. Even if the project compiles, the Lint check flags this and suggests manually upgrading the Fragment dependency to avoid runtime issues.	Would require upgrading androidx.fragment:fragment to 1.3.2 from 1.1.0	SO discussion Issue Tracker Comment - #5 , #18 , #35 Oppia PR #5582 InvalidFragmentVersionForActivityResult
MissingClass Priority - 8/10 Severity - Error Category - Correctness	The androidx.startup.InitializationProvider class is not found in the project or libraries.	This is a common issue in the community and may not have a direct approach.	IssueTracker - #1 , #2 , #3 possible solution MissingClass
NewApi Priority 6/10 Severity - Error Category - Correctness	Using APIs introduced in newer Android versions in code that targets older versions, potentially causing runtime crashes	Currently, this error is being triggered for: • <code>forEach</code> • <code>computeIfAbsent</code> • <code>getOrDefault</code> • <code>android:autofillHints</code> To resolve the issue, we may - Replace the new API with equivalent logic that works on the min SDK - Guard the call with an API level if-else check.	NewApi forEach() issue getOrDefault() issue Oppia PR #5190 Example Fix
UseRequireInsteadOfGet Priority 6/10 Severity - Error Category - Correctness	The error occurs because <code>get____()</code> methods return null when the requested object isn't available, leading to potential NullPointerExceptions with unclear error messages.	Replace activity!! with <code>requireActivity()</code> in <code>LogoutDialogFragment.kt</code> Replace arguments!! with <code>requireArguments()</code> in <code>SurveyFragment.kt</code>	UseRequireInsteadOfGet requireArguments requireActivity Example Fix
UseAppTint Priority 1/10	ImageView or ImageButton uses <code>android:tint</code> instead of	Use <code>app:tint</code> instead of <code>android:tint</code>	UseAppTint Example Fix

Severity - Error Category - Correctness	app:tint		
SelectableText Priority - 7/10 Severity - Warning Category - Usability	TextViews, which are likely to display data, should be selectable for users to copy data.	<textView> should specify android:textIsSelectable="true".	SelectableText Github Discussion SO related to Selectable text - #1 , #2 Example Fix
Typos Priority 7/10 Severity - Warning Category - Correctness	Any words in string definitions that appear to be likely misspellings will be flagged.	Review flagged words and correct genuine typos. If a word is intentional, suppress the warning using tools:ignore.	Typos Example Fix
OldTargetApi Priority 6/10 Severity - Warning Category - Correctness	Having an outdated targetSdkVersion or targetApi annotation.	Testing and updating the targetSdkVersion (latest SDK version being 36.0.0) to ensure compatibility.	OldTargetApi Oppia PR #5604 - Upgraded builds to target SDK 34 Migration
UnusedAttribute Priority 6/10 Severity - Warning Category - Correctness	Caused when attributes are set in XML files that were introduced in a version newer than the oldest version targeted by your application	These issues can be resolved by using support libraries, or another way is to use conditional resource qualifiers, which involve creating separate layout files based on the minimum API level that supports the attribute.	UnusedAttribute Oppia PR #5662 - Fixed UnusedAttribute Lint issues
ObsoleteSdkInt Priority 6/10 Severity - Warning Category - Performance	Occurs when a resource folder is unnecessary because the app's minSdkVersion already meets or exceeds that API level.	Consolidate the resources from the res/values-v21 directory into the main values directory	ObsoleteSdkInt Example Fix
RtlSymmetry Priority 6/10 Severity - Warning Category - Internationalization	This warning occurs because paddingStart is defined without paddingEnd, which can cause layout issues in right-to-left (RTL)	Ensure that when using android:paddingStart or android:marginStart, also define the corresponding android:paddingEnd or	RtlSymmetry Example Fix

	languages.	android:marginEnd Currently being triggered from create_profile_fragment.xml , which would require setting android:paddingEnd and testing for asymmetry issues in RTL layouts.	
UnknownNullness Priority 6/10 Severity - Warning Category - Interoperability	The issue occurs when a method parameter or return type in Java lacks explicit nullability annotations (@NonNull or @Nullable)	Add nullability annotations (@NonNull or @Nullable) from androidx.annotation. Currently only being triggered for Java code in the app module.	UnknownNullness Example Fix
StringFormatCount Priority 5/10 Severity - Warning Category - Correctness	This occurs due to a mismatch between the number of format specifiers in a string and the arguments provided when formatting. The issue normally happens when the format strings used have different numbers of arguments defined across translations.	Ensure that the number of format specifiers (%s, %d, etc.) matches the number of arguments passed. Also, add tests to cover each case where a mismatch could occur, as discrepancies may lead to app crashes. The errors are primarily in the Swahili translations where format specifiers contain spaces ("% 1 \$ s") and in the Arabic translations, where the format specifier positions are incorrect or are inconsistent	SO discussion Oppia issue #4562 , #5637 Oppia PR #5524 StringFormatCount Example Fix
UnusedResources Priority 3/10 Severity - Warning Category - Performance	Resources (drawables, layouts, strings, etc.) that aren't referenced anywhere in the code.	Remove unused resources or suppress using <code>tools:ignore="UnusedResources"</code>	UnusedResources SO discussion Example Fix
LabelFor Priority 2/10 Severity - Warning Category - Accessibility	Input fields are lacking associated labels for accessibility.	Use either android:hint for input fields or associate a label with android:labelFor to link a TextView with its corresponding view. When using android:labelFor, ensure to provide an	LabelFor Example Fix

		android:text or an android:contentDescription.	
UselessLeaf Priority 2/10 Severity - Warning Category - Performance	Layout elements that don't impact the UI should be removed. This includes views with no children, no ID, no styles or visual attributes, and those not involved in data binding, transitions, or animations.	Review flagged layout files and remove unnecessary leaf views, or add an ID to suppress the warning Currently being triggered in: downloads_tab_fragment.xml drawer_fragment.xml updates_tab_fragment.xml	UselessLeaf Example Fix
UselessParent Priority 2/10 Severity - Warning Category - Performance	A layout with children that have no siblings is not a scrollview or a root layout, and does not have a background	Identify and remove unnecessary wrapper layouts; use layout attributes on child views to achieve the same positioning/styling Currently being triggered from: pin_password_activity.xml revision_card_fragment.xml	UselessParent Example Fix
DuplicateStrings Priority 2/10 Severity - Warning Category - Performance	Duplicate strings can make applications unnecessarily larger. This lint issue looks for duplicate strings, including differences where the only variation is in capitalization.	Merge the duplicate strings into a single entry in the strings.xml file and use the corresponding string resource name in all relevant places.	DuplicateStrings Oppia issue #3384
UnusedIds Priority 1/10 Severity - Warning Category - Performance	This resource ID definition appears not to be needed since it is not referenced from anywhere.	Having ID definitions, even if unused, isn't harmful—they aid layout/menu editing. To resolve this lint issue, either remove the unused IDs or suppress it with tools:ignore.	UnusedIds Example Fix

Issues will be addressed over time or tracked under a larger Android Lint issue collection (similar to [#5169](#)) if unresolved.

Third-Party Libraries

The version of third-party libraries can be determined by adding 23 to the Lint version. Given that the Lint version is **8.0.0** (as per the analysis above), the corresponding library version would be **31.0.0**. [\[source\]](#)

No.	Third-party library name and version	Link to third-party library	Why it is needed	License (if third-party library)	[Android only] Min / target / max SDK version that the library supports
1	Lint	lint-31.0.0	This is the main executable JAR for the Android Lint tool.	Apache-2.0 license	
2	Lint-checks	lint-checks-31.0.0	Includes a collection of pre-built rules that analyze code for common issues.	Apache-2.0 license	
3	Lint-api	lint-api-31.0.0	API to build lint checks	Apache-2.0 license	
4	Lint-model	lint-model-31.0.0	Defines the build system model for Lint, facilitating the integration between Lint and Bazel	Apache-2.0 license	

“Service” Dependencies

This project adds no new service dependencies

Key High-Level and Architectural Decisions

Decision 1: How To Invoke the Lint CLI Tool?

We have considered the following alternatives:

1. **Command line utilities** - Requires Android SDK and Runs lint via its CLI tool, dynamically resolving SDK JAR paths.
2. **Maven Dependencies** - Uses Bazel-managed Maven dependencies (e.g., lint-api, lint-checks) and Invokes lint via reflection on classes loaded by Bazel’s runtime classpath.

Among these, we believe that Maven Dependencies is the best approach because:

- **Dependency** - Eliminates the need for local Android cmdline-tools setup.
- **Version Control** - The lint version is pinned via Maven, ensuring reproducibility.
- **Bazel Integration** - Leverages Bazel’s dependency resolution and caching.

The above approaches are contrasted in detail in the following table:

Aspect	Maven Dependencies	Command Line Utilities
Setup Complexity	No setup beyond Bazel is required.	Requires manual Android cmdlint-tool installation.
Version Control	Reproducible builds; Bazel fetches the exact lint version specified.	Depends on the lint version that comes with cmdline-tools.
Integration	Tightly integrated with Bazel's dependency and build systems.	Decoupled from Bazel's build graph.
Control	Direct access to lint internals via reflection, enabling advanced configuration.	Limited control over lint's internal state.
Performance	Faster in-process execution but minor reflection overhead.	Slightly slower due to JVM startup overhead for each run.

Risks and mitigations

Potential Risk	Mitigation
Lint's internal classes/methods may change in updates, breaking reflection.	Create targeted unit tests with a minimal fake codebase that triggers each failure case individually. Monitor lint release notes and pin to a stable minor version.
New lint versions may introduce crashes or false positives.	Test new lint versions in a controlled branch before upgrading.

Decision 2: Which Report Format To Utilize?

We have considered the following alternatives:

1. **XML** - Parse the generated XML lint reports to determine Pass/Fail status
2. **SARIF** - Parse the generated SARIF lint reports to determine Pass/Fail status

Among these, we believe that XML is the best approach because:

- **Well Supported** - Kotlin and Java provide robust XML parsing libraries (DOM, SAX), making it easy to process reports without additional dependencies.
- **Ease of Debugging** - XML is human-readable, structured, and hierarchical, making it easier to inspect and debug
- **Consistency** - XML parsing is already used in other scripts, ensuring uniformity and reducing the need for different parsing approaches.

The above approaches are contrasted in detail in the following table:

Aspect	XML	SARIF
Adoption in Codebase	Already used in other scripts, ensuring consistency.	Not used in existing scripts.
Parsing Libraries	Java & Kotlin have well-supported XML parsers (DOM, SAX).	SARIF requires JSON parsing, adding dependencies.
Readability & Debugging	XML is human-readable and easier to debug.	SARIF JSON can be deeply nested, making debugging harder.
Performance	Slightly slower due to its hierarchical nature, but optimized with SAX.	Faster JSON parsing, but a complex SARIF structure may offset the advantage.

Risks and mitigations

Potential Risk	Mitigation
Potential performance concerns with XML parsing	Use SAX parsing for efficiency instead of DOM when dealing with large files.
SARIF might be required in the future	If SARIF adoption is needed, an abstraction layer can be introduced later.

Key Objectives for each Milestone

Milestone	Key Objective
Milestone 1	Developers will be able to: • Detect Android lint issues without relying on Gradle using automated scripts. • Supports exemptions via a textproto-based override system. • Refer to the clear documentation on how to use the script and update exemptions. • Run lint checks automatically on every change to the PR or develop branch.
Milestone 2	Oppia-Android codebase will have: • All 'error/fatal' category lint issues fixed to improve code correctness and maintainability. • Resolved key 'warning' category issues (e.g., RtlSymmetry, ObsoleteSdkInt, etc.) • All unresolved issues are tracked with corresponding GitHub issues, providing clarity for future fixes.

Implementation Approach

Domain Objects

N/A

User Flows (Controllers and Services)

N/A

[Android only] UI changes

This project primarily involves script and CI modifications aimed at enhancing the developer workflow. While it does not directly modify the UI, resolving lint issues based on analysis reports may lead to UI changes.

The following are a few potential changes that may be required while resolving issues. Each change should be carefully tested to ensure it does not introduce unintended UI modifications or regressions.

Issue	Possibility	Potential UI impacts	Affected Files
SelectableText	Yes	Makes text selectable, which alters user interaction.	[List of Textview Changes]
ObsoleteSdkInt	Possible	Consolidating resources from res/values-v21 into the main values directory might affect styling if those resources contain different values.	res/values-v21
RtlSymmetry	Possible	resolving this warning by adding paddingEnd may result in UI changes in right-to-left (RTL) layouts, but not in left-to-right (LTR) layouts.	Create profile fragment.xml - use symmetric padding for RTL-LTR layouts.
LabelFor	Not visually, but it affects UX	It affects accessibility more than UI, but may indirectly affect TalkBack users' experience.	AutoCompleteTextView in: onboarding_app_language_selection_fragment.xml audio_language_selection_fragment.xml
Typos	Yes	Affects string resources and changes string values to avoid misspellings.	Entries in strings.xml onboarding_language_activity_title audio_language_fragment_subtitle third_party_dependency_name_1

--	--	--	--

[Android only] Test data changes

This project will not require modifications to test data. However, it will involve creating a temporary test folder (similar to [TextViewStyleCheckTest](#)) containing source and resource files (*.kt, *.java, *.xml) with deliberate lint issues, along with skeleton classes that define only the relevant library signatures. Specifically, it will include only the subset necessary for testing. For example, to test for RtlHardcoded and RtlEnabled lint issues:

tempDir/app/src/main/res/layout/rtl.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Test"
        android:layout_gravity="left"/> <!-- This should be "start" instead of "left" -->

</LinearLayout>
```

tempDir/app/src/main/AndroidManifest.xml

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.app">

    <application
        android:theme="@style/AppTheme"
        android:label="ExampleApp">
        <!-- Missing android:supportsRtl="true" or "false", triggering RtlEnabled warning -->
    </application>

</manifest>
```

Test setups regarding different lint issues can be referenced from [\[source\]](#)

[Android only] Testing library changes

The tests in this project will require the [lint-test-31.0.0](#) library to verify that the script correctly identifies specific categories of lint issues. This will be achieved by comparing the script's output with the expected results from the lint() API. [\[lint-test Docs\]](#)

```

@Test
fun testLintScript_detectsSameIssuesAsLintTest() {
    // Step 1: Run the custom lint script on simulated files with known issues
    val scriptOutput = AndroidLintRunner(
        /* files containing lint issues */
    )

    // Step 2: Run the lint-test framework on the same test files
    lint()
        .files(
            // Simulated test files matching the script input
        )
        .issues(
            // List of issues expected to be detected
        )
        .run()
        .expect(scriptOutput) // Ensure the script catches similar issues
}

```

[Android only] Script & CI changes

```

-scripts/
├── assets/
│   └── lint_issue_exemptions.textproto
├── src/java/org/oppia/android/scripts/
│   └── lint/
│       ├── AndroidLintRunner.kt
│       ├── BUILD.bazel
│       ├── LintAnalysisReporter.kt
│       └── LintProjectDescription.kt

```

Deliverable 1.1:

The developer triggers a script that wraps the lint command line tool

AndroidLintRunner:

This will serve as the core of the lint analysis script, invoking the Lint tool to generate an XML report. It will function as the central entry point for the entire analysis workflow.

- Timeouts to prevent hangs and async code execution can be handled while invoking the Lint tool, similar to [RunCoverage](#).
- The arguments list can be implemented similarly to [AndroidLintRunner](#) with the relevant flag required for the tool's configuration.


```

/**
 * Main entry point for the Android Lint script.
 *
 * @param args command-line arguments
 */
fun main(vararg args: String) {
}

/**
 * Runs the Android lint tool to generate reports.
 */
class AndroidLintRunner {

    /**
     * Prepares arguments for Lint and invokes the tool.
     *
     * @param repoPath path to the repository
     * @param targetModules list of modules to lint, defaults to common modules
     *                      (app, utility, domain, data, testing) if empty
     */
    fun runLint(repoPath: String, targetModules: List<String>) {
        // Prepare lint arguments
        // Call invokeLint with arguments
        // Print results based on exit code
    }

    /**
     * Asynchronously invokes the Android Lint tool with provided arguments.
     *
     * @param args lint command arguments
     * @return exit status of the lint command
     *
     * Exit Status:
     * 0 -> Success
     * 1 -> Lint issues found
     * 2 -> Incorrect usage of the Lint command.
     */
    private suspend fun invokeLint(args: Array<String>): Deferred<Int> {
    }
}

```

LintProjectDescription:

This utility will handle the generation of project-description.xml file, utilizing the BazelClient for file path collection ([\[Link to sample XML with all tags\]](#)) ([\[Sample project description file\]](#))

- XML file writing logic using DocumentBuilderFactory for generating project-description.xml can be referenced from [AndroidLintProject](#) used in rules_android_lint.

- [attrs.bzl](#) also provides insight into the sets of file paths that need to be collected for configuration, and it uses [collect_aar_outputs_aspect.bzl](#) to retrieve and manage AAR artifacts related to the module's dependencies.
- Another implementation that could be referenced while generating the project-description can be [\[Link to Github\]](#).
- The Lint tool processes this XML file through the [ProjectInitializer](#) and can be used as a reference for the expected XML structure.
- The main key points to consider while implementing this section would be to
 - Make the XML file generation resilient to project structure changes.
 - The utility should fail if the chances of False positives are high.
 - Use the current Gradle build files to piece together the context the Lint tool would require via the project description for better accuracy.
- Source (*.java,*.kt) and resource (*.xml) file paths will be collected for module configurations(for <source> and <resource> tags).
- Manifests and <module>/proguard-rules.pro file paths will be collected (for <manifest> and <proguard> tags)

```
/**
 * Responsible for generating the project description XML required by the Android Lint
 * tool.
 */
class LintProjectDescription {

    /**
     * Collects file paths and generates a project description XML file for the lint tool.
     *
     * @param bazelClient client used to query file paths from Bazel
     * @param outputFile destination file where the project XML will be written
     */
    fun generateProjectDescriptionXml(
        bazelClient: BazelClient,
        outputFile: File,
        repoRoot: String
    ) {
        // Set project tag and root directory in XML
        // For each module (/app,/utility,/domain,/testing,/data):
        //   -> Set module metadata like name, compile-sdk-version, javaLanguage, kotlinLanguage
        //   -> Collect file paths present in <module>/src/ (*.java, *.kt, *.xml)
        //   -> Collect file paths for module manifests, proguard-rules.pro
        //   -> Use bazelClient to gather build-related paths
        // Build XML structure with collected paths
    }

    /**
     * Collects source files for a target module
     *
     * @param targetModule target module for which source files are collected
     * @return list of file paths for source files within the module.
     */
    private fun collectSourceFilePaths(targetModule: String): List<String> {
```

```

    // Collect source file paths in <module>/src (*.java, *.kt)
}

/**
 * Collects all XML resource file paths for the given module.
 *
 * @param targetModule target module for which resource files are collected
 * @return list of resource file paths
 */
private fun collectResourceFilePaths(targetModule: String): List<String> {
    // Collect resource file paths in <module>/src/main/res (*.xml)
}

/**
 * Extracts the AAR file to a temporary directory and generates an XML tag for it.
 *
 * @param aarFilePath the path to the AAR file
 * @param tempDir base directory to extract AARs to
 * @return an XML tag string in the format: <aar file="..." extracted="...">
 */
private fun generateAarTag(aarFilePath: String, tempDir: File): String {
}

/**
 * Parses the merged manifest output from Bazel and generates XML tags.
 *
 * @param aqueryOutput output text from retrieveMergedManifest
 * @return list of XML tags like: <merged-manifest file="...">
 */
private fun generateMergedManifestTags(aqueryOutput: String): List<String> {
}

/**
 * Writes a DOM XML Document object to a specified file.
 *
 * @param document the XML document to write
 * @param outputFile destination file for the XML output
 */
private fun writeDocumentToFile(document: Document, outputFile: File) {
    // Use XML transformer to write document to file system
}
}

```

BazelClient:

This introduces new functions in BazelClient to collect file paths related to a module using Bazel's query language for the project description. Using Bazel's query language enhances the resilience of project description generation against changes in project structure.

- Collect AAR and classpath JAR files (for <aar> and <classpath> tags).
- Collect the file paths of all modules that are dependencies of the target module (for <dep> tag).

- Collect merged manifests (for <merged-manifest> tag).

```
class BazelClient() {
// Existing methods and properties...
/**
 * Retrieves all transitive output files generated by building the given Bazel target.
 *
 * @param bazelTarget Bazel target module for which files are to be collected
 * @return list of file paths related to the Bazel Module
 */
fun retrieveTransitiveOutputFiles(bazelTarget: String): List<String> {
    // Executes: bazel cquery "deps(//$bazelTarget:*)" --output=files
}

/**
 * Retrieves all transitive module dependencies of the given Bazel target.
 *
 * @param bazelTarget Bazel target module whose dependencies are to be resolved
 * @return list of Bazel target labels that are part of the dependency graph
 */
fun retrieveModuleDependencies(bazelTarget: String): List<String> {
    // Executes: bazel query "kind(android_library, deps(//$bazelTarget))"
}

/**
 * Retrieves merged AndroidManifest information for the specified Bazel target.
 *
 * @param bazelTarget Bazel target module for which manifest merge info is to be collected
 * @return text representation of the manifest merge actions, including merged file paths
 */
fun retrieveMergedManifest(bazelTarget: String): String {
    // Executes: bazel aquery 'mnemonic("ManifestMerger", deps(//$bazelTarget:*))' --output=text
}
}
```

Deliverable 1.2:

Enhance the script with an exemption system and ensure accurate reporting of lint issues

Exemption Handling:

script_exemptions.proto:

This defines the message format that will be used in the textproto exemption list

- Using enums for Lint issue IDs offers type safety but requires updating the proto and dependent code whenever new issues are introduced. To balance safety and maintainability, we include only the subset of Lint issues that are actually encountered or likely to be triggered and explicitly fail on any unsupported issues.
- The ID 'ISSUE_UNSPECIFIED' has been reserved for Unspecified issues for error handling [\[source\]](#)

```

message AndroidLintExemptions {
    repeated AndroidLintExemption android_lint_exemption = 1;
}

message AndroidLintExemption {
    string exempted_file_path = 1; // File path for which exemptions are specified.
    repeated LintIssueId lint_issue_id = 2; // List of issues to ignore for this path.
}

// Only includes known, expected lint issues.
// Script should fail on unknown issues, prompting updates to this
enum LintIssueId {
    ISSUE_UNSPECIFIED = 0; // Case when no ID is specified
    OBSOLETE_LAYOUT_PARAM = 1;
    UNUSED_RESOURCES = 2;
    NEW_API = 3;
    // Rest of the issues...
}

```

Changes to “script_assets.bzl” will be made to facilitate the conversion of the textproto list to Binary, similar to [todo_asset_list](#)

LintAnalysisReporter:

This utility includes the exemption handling and result XML parsing logic to interpret results and log lint analysis reports

- Parsing of the result XML will be done similarly to [TextViewStyleCheck](#).
- Loading of textproto exemptions can be referenced from [KdocValidityCheck/ToDoOpenCheck](#).

```

/**
 * Handles exemption processing, lint result parsing, and reporting for Android Lint
 * analysis.
 */
class LintAnalysisReporter {

    /**
     * Parses the given lint result XML file, filters issues based on exemptions,
     * logs relevant issues and detects any outdated exemptions.
     *
     * @param resultFile XML file generated by Android Lint containing analysis results
     */
    fun parseResultXml(resultFile: File) {
        // Parse lint result XML and collect reported issues
        // Prompt to update LintIssueId Enum if a new issue ID is found
        // Load exemptions and filter issues to report to the user
    }
}

```

```

}

/**
 * Logs exemptions that are no longer relevant based on the current lint results.
 *
 * @param redundantExemptions list of unused exemptions
 * @param lintExemptionTextProtoFilePath path to the exemption textproto source file
 */
private fun logRedundantExemptions(
    redundantExemptions: List<AndroidLintExemption>,
    lintExemptionTextProtoFilePath: String
) {
}

/**
 * Converts the numeric enum value to the corresponding string identifier.
 *
 * @param enumValue the numeric value from the LintIssueId enum
 * @return PascalCase string identifier of the corresponding lint issue
 */
private fun convertIssueIdToString(enumValue: Int): String {
    // Mapping of enum ID to Issue ID
}

/**
 * Loads Android lint exemptions from a compiled textproto binary file.
 *
 * @param pathToProtoBinary path to the binary proto file
 * @return parsed AndroidLintExemptions object
 */
private fun loadExemptions(pathToProtoBinary: String): AndroidLintExemptions {
}

/**
 * Writes a DOM XML Document object to a specified file.
 *
 * @param document the XML document to write
 * @param outputFile destination file for the XML output
 */
private fun writeDocumentToFile(document: Document, outputFile: File) {
    // Transform DOM document to XML and write to file
}
}

```

With respect to the project, the task for populating the textproto exemption file (under “scripts/assets”) can be automated by utilizing the `parseResultXml` function in `LintAnalysisReporter`.

Deliverable 1.3:

Integration of lint analysis into the CI workflow to run for changes to PRs and the develop branch.

CI Integration:

Changes to [static_checks.yml](#) and [static_checks.sh](#) will be made to facilitate CI Integration for project-level lint analysis.

```
bazel run //scripts:lint_analysis_check -- $(pwd) <path to proto binary>
```

Relevant test PRs demonstrating successful execution of the workflow on changes to PRs and develop will be provided for testing of changes.

Documentation changes

As this project introduces new tooling for Android lint checks, it necessitates comprehensive documentation for developers. I plan to make the following key additions to the project's wiki:

- **Introduce a Wiki Page for Android Lint Analyses:** Create a comprehensive wiki page detailing the usage of the new Android Lint script. Include step-by-step guides on running the script, interpreting results, and resolving common lint issues to help developers use it effectively.
- **Explain the Exemption System and Configuration:** Detail the structure of exemptions in the textproto format, including guidance on when and how to add them. Explain the rationale behind the exemption system and outline best practices for its effective use.
- **Document Categories of Lint Issues:** Create comprehensive documentation for each lint category being addressed in the project. For each category, include:
 - Description of related common issues and why they matter.
 - Example of problematic code and its corrected version.
 - Common patterns and anti-patterns to watch for.

Add a reference to the [issue documentation](#) for a better understanding of individual lint issues.

- **Known Limitations and Workarounds:** Document any limitations of the Android Lint script and provide workarounds for false positives or negatives. Additionally, explain how the exemption system can be used to handle edge cases effectively.
- **Document the Structure of project-description.xml:** Provide detailed documentation on the structure and purpose of the project-description.xml file used by the lint tool. Offer examples to help developers correctly configure and maintain this file.
- **CI Integration Documentation:** Explain how Android lint checks are integrated into the CI pipeline and provide guidance on interpreting lint failures in CI builds.

This documentation will ensure that developers can effectively use the new Android lint infrastructure, understand how to address lint issues properly, and contribute to maintaining code quality across the codebase.

Testing:

Below is an overview of test names that would be added for specific key functionalities/code changes. These are not actual test names, but give a brief idea.

Functionality	Test Names	Purpose
AndroidLintRunner:	testRunLint_completesSuccessfully testRunLint_failsWithError	Verify that the script executes successfully

Script Execution	testRunLint_processTimesOut_failsWithError testInvokeLint_returnsSuccessStatus testInvokeLint_returnsErrorStatus testInvokeLint_validArguments testInvokeLint_lintInternalApisChanged_gracefullyHandled testLintInvocation_classNotFound_failsWithMeaningfulError	- Reporting issues at project-level Verify that lint returns valid exit status codes - Successful execution - 0 - Lint issues found - 1 - Incorrect Usage - 2 Verifies that lint is configured with valid arguments for report generation Ensures clear error messaging when required classes can't be found for lint invocation to validate reflection.
LintAnalysisReporter: Result XML Parsing	testParseResultXml_parsesSuccessfully testParseResultXml_reportsErrorsCorrectly testParseResultXml_reportsWarningsCorrectly testParseResultXml_exemptsIssuesCorrectly testParseResultXml_promptsNewIssues	Verify that the XML parsing - Successfully parses the XML files - Report errors correctly with a detailed description - Reports warnings correctly with a detailed description - Does not report issues present in the exemption list - Keeps track of newfound issues to prompt about Enum updates
LintAnalysisReporter: Exemption Loading	testLoadExemptions_loadsCorrectly testLogRedundantExemptions_logsCorrectly	Verify that exemptions - Are loaded correctly from the textproto file - Redundant exemptions are logged correctly, prompting removal.
LintProjectDescription: Project Description XML Generation	testProjectDescriptionXml_validModules_generatesCorrectXml testProjectDescriptionXml_validFilePaths	Verify that the project description XML - Follows a valid XML structure - Has all modules well defined in the XML with their source and resource file paths. - Defines all dependencies, libraries, AAR artifacts, and JAR files for global and module-level configurations.

Metrics Plan

Event (see PRD)	Event parameters (see PRD)	Do we already record the event + parameters? • If so, please link to the corresponding code on
-----------------	----------------------------	---

		GitHub. If not, describe the changes needed to do so.
N/A	N/A	N/A

Implementation Plan

Community Bonding Period:

I consider the community bonding period crucial for this project as it will allow me to identify areas for improvement and ensure alignment with the project's goals.

During this period, I plan to:

- Clarify any ambiguities in my proposal to ensure alignment with project goals.
- Identify areas where I need to improve my skills.
- Understanding and improving the lint tool configurations in detail to best suit the codebase, as accurate reporting is a critical aspect of this project.
- Begin building a prototype, focusing on delivering a smooth and efficient workflow in the coding period.

Milestone Table

Milestone 1

Key objective for this milestone:

Developers will be able to:

- Detect Android lint issues without relying on Gradle using automated scripts.
- Supports exemptions via a textproto-based override system.
- Run lint checks automatically on every change to the PR or develop branch.
- Refer to the clear documentation on how to use the script and update exemptions.

No.	Description of PR / action	Prerequisite steps	Target date	
			PR creation	PR merge
1	Introduce the AndroidLintRunner script to - Invoke lint and generate reports with minimal configuration for the project - Add tests for script execution and lint invocation.	None	02-06-2025	06-06-2025
2	Introduce the LintAnalysisReporter to - parse and extract XML reports - Add tests to validate the parsing logic.	None	04-06-2025	10-06-2025
3	Introduce the LintProjectDescription utility to - Generate "project-description.xml" - Execute Bazel queries for file path collection.	None	08-06-2025	22-06-2025

	- Add tests to validate correct bazel query execution. - Add tests validating XML file generation.			
4	Enhance the AndroidLintRunner script to - Generate lint reports with finalized configurations. - Add tests validating report generation logic and report accuracy.	1,2,3	20-06-2025	25-06-2025
-	<i>At this point, the lint analysis is functional, and report logs can be tested for accuracy.</i>			
5	Enhance LintAnalysisReporter to - populate issues in textproto. - load exemptions from the textproto file - generate reports with exemption support. - Implement tests to validate the lint reports for the exemption system.	5	23-06-2025	27-06-2025
	<u>Demo to PMs/stakeholders:</u> • Execution of the Lint Analyses script reporting issues in the codebase. • Support for the textproto exemption system.	6	27-06-2025	
6	Enhance static_checks.yml and static_checks.sh to facilitate CI integration of the script	6	25-06-2025	27-06-2025
7	Create a Wiki page for the Android Lint script.	None	26-06-2025	29-06-2025
	<u>Demo to PMs/stakeholders:</u> • Lint Analysis workflow running on changes to PRs and develop • Wiki for Lint Analyses	7,8	29-07-2025	
8	Resolve issues with severity 'Error', - UseRequireInsteadOfGet - UseAppTint - Add relevant tests.	None	27-06-2025	30-06-2025
10	Buffer Time: To work on reviews and code changes.	None	30-06-2025	04-06-2025

Milestone 2

Key objective for this milestone:

Oppia-Android codebase will have:

- All 'error/fatal' category lint issues fixed to improve code correctness and maintainability.
- Resolved key 'warning' category issues (e.g., RtlSymmetry, ObsoleteSdkInt, etc.)
- All unresolved issues are tracked with corresponding GitHub issues, providing clarity for future fixes.

No.	Description of PR / action	Prerequisite steps	Target date	
			PR creation	PR merge
1	Resolve issues with severity 'Error', - NewApi - Add relevant tests.	None	26-07-2025	01-08-2025
2	Resolve issues with severity 'Fatal' - InvalidFragmentVersionForActivityResult - Add relevant tests.	None	30-07-2025	03-08-2025
3	Resolve issues with severity 'Error', - MissingClass - Add relevant tests.	None	01-08-2025	07-08-2025
	<u>Demo to PMs/stakeholders:</u> • All issues with the severity 'Error/Fatal' are resolved.	M1 - 10 M2 - 1,2,3	07-08-2025	
4	Resolve issues with severity 'Warning' - UnusedResources - Add relevant tests.	None	05-08-2025	12-08-2025
5	Resolve issues with severity 'Warning' - DuplicateStrings - Add relevant tests.	None	09-08-2025	13-08-2025
6	Resolve issues with severity 'Warning' - SelectableText - UnknownNullness - Add relevant tests.	None	11-08-2025	15-08-2025
7	Resolve issues with severity 'Warning' - StringFormatCount - UnusedAttribute - LabelFor - Add relevant tests.	None	13-08-2025	17-08-2025
8	Resolve issues with severity 'Warning' - OldTargetApi - Add relevant tests.	None	15-08-2025	22-08-2025
9	Resolve issues with severity 'Warning' - UnusedIds - Typos - RtlSymmetry	None	20-08-2025	24-08-2025

	- UselessParent - Add relevant tests.			
10	Resolve issues with severity 'Warning' - UselessLeaf - ObsoleteSdkInt - Add relevant tests.	None	23-08-2025	27-08-2025
11	File GitHub issues to track unresolved lint issues	8	26-08-2025	
	<u>Demo to PMs/stakeholders:</u> • Key issues with a severity 'Warning' are resolved. • GitHub issues filed for unresolved issues	4-11	27-08-2025	

Feature Flags

#	Name of feature flag	What happens when the flag is turned on	Needs sync between Android / Web?	Key stakeholders
1.	N/A	N/A	N/A	N/A

Future Work

1. Address any remaining unresolved lint issues from the project and proactively reduce the occurrence of lint violations in the codebase.
2. Provide ongoing documentation updates on best practices for resolving lint issues and maintaining exemption files.
3. Regularly review and update the lint script to incorporate new lint categories, workflows, and troubleshooting tips.
4. Continuously monitor and address any gaps in lint issue analysis.